

Протокол

Автор: Андрій Білий <https://github.com/bilyilandrii>

Репозиторіум (micro_mq): https://github.com/bilyilandrii/soft_arch_hw_bilyi.git

Налаштування

Для виконання цієї лаби я використовуватиму Kafka Message Queue та буду спілкуватися з нею через асинхронного клієнта: aiokafka. Для того щоб вона запрацювала я підготував отакі настройки в docker-compose.yml:

```
kafka:
  image: apache/kafka:latest
  ports:
    - "9092:9092"
  environment:
    # Using Kraft mode to avoid needing Zookeeper
    KAFKA_NODE_ID: 1
    KAFKA_PROCESS_ROLES: broker,controller
    KAFKA_LISTENERS:
PLAINTEXT://0.0.0.0:9092,CONTROLLER://0.0.0.0:9093
    KAFKA_ADVERTISED_LISTENERS: PLAINTEXT://kafka:9092
    KAFKA_CONTROLLER_LISTENER_NAMES: CONTROLLER
    KAFKA_LISTENER_SECURITY_PROTOCOL_MAP:
CONTROLLER:PLAINTEXT,PLAINTEXT:PLAINTEXT
    KAFKA_CONTROLLER_QUORUM_VOTERS: 1@localhost:9093
    KAFKA_OFFSETS_TOPIC_REPLICATION_FACTOR: 1
    KAFKA_TRANSACTION_STATE_LOG_REPLICATION_FACTOR: 1
    KAFKA_TRANSACTION_STATE_LOG_MIN_ISR: 1

  healthcheck:
    test:
      [
        "CMD-SHELL",
        "/opt/kafka/bin/kafka-topics.sh --bootstrap-server
localhost:9092
        --list",
      ] # list all existing topics via shell command, either 0 or 1+
    interval: 5s
    timeout: 5s
    retries: 5
```

Ініціалізація відбувається через Kraft, тобто одна нода може бути контролером і брокером одночасно. Це забирає потребу додаткового серверу менеджменту як ZooKeeper. Оскільки Кафка думає що ми будемо піднімати її в production середовище,

потрібно трішки зменшити її базові рівні реплікації, явно вказати які ролі цієї ноди та як до неї доступитися.

Після запуску можемо побачити що всі сервіси підняті: facade, counter, 3 loggings, 3 hazelcasts, postgres, kafka, config і додатково management-center:

CONTAINER ID	IMAGE	COMMAND	CREATED	STATUS	PORTS	NAMES
fa7d240474a2	counter-service:latest	"uv run fastapi run _"	14 seconds ago	Up 7 seconds		hws-counter_service-1
01a536913fcc	facade-service:latest	"uv run fastapi run _"	14 seconds ago	Up 7 seconds	0.0.0.0:8080->8080/tcp, [::]:8080->8080/tcp	hws-facade-1
724ee81d0304	logging-service:latest	"uv run fastapi run _"	15 seconds ago	Up 3 seconds		hws-logging3-1
a2e86f41a98a	logging-service:latest	"uv run fastapi run _"	15 seconds ago	Up 3 seconds		hws-logging1-1
fd2530bb44c	logging-service:latest	"uv run fastapi run _"	15 seconds ago	Up 3 seconds		hws-logging2-1
48f4f01122ad	postgres:15-alpine	"docker-entrypoint.s..."	15 seconds ago	Up 14 seconds (healthy)	0.0.0.0:5432->5432/tcp, [::]:5432->5432/tcp	hws-postgres_db-1
4e1983303c62	apache/kafka:latest	"/__cacert_entrypoint..."	15 seconds ago	Up 14 seconds (healthy)	0.0.0.0:9092->9092/tcp, [::]:9092->9092/tcp	hws-kafka-1
a89b1b6eeccd	config-server:latest	"uv run fastapi run _"	15 seconds ago	Up 14 seconds	0.0.0.0:8080->8080/tcp, [::]:8080->8080/tcp	hws-config_server-1
b6aff32f9b30	hazelcast/hazelcast:5.3	"hz start"	15 seconds ago	Up 14 seconds (healthy)	0.0.0.0:5703->5701/tcp, [::]:5703->5701/tcp	hws-hazelcast3-1
ea2e3a1289ed	hazelcast/hazelcast:5.3	"hz start"	15 seconds ago	Up 14 seconds (healthy)	0.0.0.0:5702->5701/tcp, [::]:5702->5701/tcp	hws-hazelcast2-1
3a0cac045f3	hazelcast/management-center:5.3	"bash ./bin/mc-start..."	15 seconds ago	Up 14 seconds	0.0.0.0:8081->8080/tcp, [::]:8081->8080/tcp	hws-management-center-1
5ddb65de11b	hazelcast/hazelcast:5.3	"hz start"	15 seconds ago	Up 14 seconds (healthy)	0.0.0.0:5701->5701/tcp, [::]:5701->5701/tcp	hws-hazelcast1-1

/health endpoint це підтверджує:

Curl

```
curl -X 'GET' \
  'http://localhost:8080/health' \
  -H 'accept: application/json'
```

Request URL

http://localhost:8080/health

Server response

Code	Details
------	---------

200

Response body

```
{
  "status": "ok",
  "config_server": "ok",
  "kafka": "ok",
  "counter_services": {
    "http://counter_service:8080": {
      "status": "ok",
      "database": "connected"
    }
  },
  "logging_services": {
    "http://logging3:8080": {
      "status": "ok",
      "hazelcast": "connected"
    },
    "http://logging1:8080": {
      "status": "ok",
      "hazelcast": "connected"
    },
    "http://logging2:8080": {
      "status": "ok",
      "hazelcast": "connected"
    }
  }
}
```

Під час запуску основних сервісів, вони будуть звертатися до конфігураційної ноди через /register endpoint:

```
await http_client.post(
    f"{CONFIG_SERVER_URL}/register",
    json={"name": "counter-service", "url": MY_URL},
)
```

де CONFIG_SERVER_URL - адреса configa, а MY_URL - адреса поточного контейнера.

З логів можете побачити що 4 сервіси були успішно записані до таблиці конфігурації:

```
Logs:

INFO    Started server process [10]
INFO    Waiting for application startup.
INFO    Application startup complete.
INFO    Uvicorn running on http://0.0.0.0:8000 ↗ (Press CTRL+C to quit)
[CONFIG] Registered instance http://counter\_service:8000 ↗ for counter-service
INFO    172.18.0.9:52822 - "POST /register HTTP/1.1" 200
[CONFIG] Registered instance http://logging3:8000 ↗ for logging-service
INFO    172.18.0.13:33844 - "POST /register HTTP/1.1" 200
[CONFIG] Registered instance http://logging1:8000 ↗ for logging-service
INFO    172.18.0.11:42954 - "POST /register HTTP/1.1" 200
[CONFIG] Registered instance http://logging2:8000 ↗ for logging-service
```

А для того щоб отримати список наявних вузлів за ім'ям сервісу можна звернутися до `/services/{name}` ендпоінту. Метадані зберігаються локально у хеш-таблиці конфігураційного сервісу. Для цього я підготував наведену функцію, яка за ім'ям сервісу вертає його URL:

```
async def get_service_urls(http_client: httpx.AsyncClient, service_name:
str) -> list:
    try:
        resp = await
http_client.get(f"{CONFIG_SERVER_URL}/services/{service_name}")
        if resp.status_code == 200:
            return resp.json().get("instances", [])
    except Exception as e:
        print(f"[FACADE] Error fetching {service_name} from config
server: {e}")
    return []
```

Також можна побачити що HZ nodes також працюють та утворили єдиний кластер:

```
Initialized new cluster connection between /172.18.0.6:60415 and hazelcast1/172
2026-04-19 09:05:10,483 [ INFO] [hz.flamboyant_blackwell.priority-generic-opera
cluster] [5.3.8]

Members {size:3, ver:3} [
    Member [hazelcast1]:5701 - c2febb77-549f-4c57-8e37-f1e4c957b226
    Member [hazelcast2]:5701 - e9944c12-e441-4675-a2a5-806d5d0e03bc this
    Member [hazelcast3]:5701 - b84baa8e-ca71-491e-8b54-808530d870b8
]

2026-04-19 09:05:10,496 [ INFO] [main] [c.h.c.LifecycleService]: [hazelcast2]:5
```

Базовий Функціонал

Роблю 10 запитів msg1-10 на id “user”.

Запити/Відповіді:

Curl

```
curl -X 'POST' \
'http://localhost:8080/process' \
-H 'accept: application/json' \
-H 'Content-Type: application/json' \
-d '{
  "user_id": "user",
  "amount": 10.2
}'
```

Request URL

http://localhost:8080/process

Server response

Code	Details
200	Response body <pre>{ "transaction_id": "1776590730744523465", "status": "Message queued successfully" }</pre>

Curl

```
curl -X 'POST' \
'http://localhost:8080/process' \
-H 'accept: application/json' \
-H 'Content-Type: application/json' \
-d '{
  "user_id": "user",
  "amount": 66
}'
```

Request URL

http://localhost:8080/process

Server response

Code	Details
200	Response body <pre>{ "transaction_id": "1776590787458214547", "status": "Message queued successfully" }</pre>

Curl

```
curl -X 'POST' \
'http://localhost:8080/process' \
-H 'accept: application/json' \
-H 'Content-Type: application/json' \
-d '{
  "user_id": "user",
  "amount": -6.2
}'
```

Request URL

http://localhost:8080/process

Server response

Code	Details
200	Response body <pre>{ "transaction_id": "1776590826581112759", "status": "Message queued successfully" }</pre>

Curl

```
curl -X 'POST' \
'http://localhost:8080/process' \
-H 'accept: application/json' \
-H 'Content-Type: application/json' \
-d '{
  "user_id": "user",
  "amount": -20
}'
```

Request URL

http://localhost:8080/process

Server response

Code	Details
200	Response body <pre>{ "transaction_id": "1776590878472449547", "status": "Message queued successfully" }</pre>

Curl

```
curl -X 'POST' \
'http://localhost:8080/process' \
-H 'accept: application/json' \
-H 'Content-Type: application/json' \
-d '{
  "user_id": "user",
  "amount": 33
}'
```

Request URL

http://localhost:8080/process

Server response

Code	Details
200	Response body <pre>{ "transaction_id": "1776590899231783501", "status": "Message queued successfully" }</pre>

Curl

```
curl -X 'POST' \
'http://localhost:8080/process' \
-H 'accept: application/json' \
-H 'Content-Type: application/json' \
-d '{
  "user_id": "user",
  "amount": 100
}'
```

Request URL

http://localhost:8080/process

Server response

Code	Details
200	Response body <pre>{ "transaction_id": "1776590918794465719", "status": "Message queued successfully" }</pre>

Curl

```
curl -X 'POST' \
'http://localhost:8080/process' \
-H 'accept: application/json' \
-H 'Content-Type: application/json' \
-d '{
  "user_id": "user",
  "amount": -50
}'
```

Request URL

http://localhost:8080/process

Server response

Code	Details
200	Response body <pre>{ "transaction_id": "1776590939634532089", "status": "Message queued successfully" }</pre>

Curl

```
curl -X 'POST' \
'http://localhost:8080/process' \
-H 'accept: application/json' \
-H 'Content-Type: application/json' \
-d '{
  "user_id": "user",
  "amount": 330
}'
```

Request URL

http://localhost:8080/process

Server response

Code	Details
200	Response body <pre>{ "transaction_id": "1776590969560479173", "status": "Message queued successfully" }</pre>

Curl

```
curl -X 'POST' \
'http://localhost:8080/process' \
-H 'accept: application/json' \
-H 'Content-Type: application/json' \
-d '{
  "user_id": "user",
  "amount": -150
}'
```

Request URL

http://localhost:8080/process

Server response

Code	Details
200	Response body <pre>{ "transaction_id": "1776590989161231209", "status": "Message queued successfully" }</pre>

Curl

```
curl -X 'POST' \
'http://localhost:8080/process' \
-H 'accept: application/json' \
-H 'Content-Type: application/json' \
-d '{
  "user_id": "user",
  "amount": 1.5
}'
```

Request URL

http://localhost:8080/process

Server response

Code	Details
200	Response body <pre>{ "transaction_id": "1776590999999999999", "status": "Message queued successfully" }</pre>

Як можемо зауважити, цього разу ми не отримуємо одразу інформацію про баланс користувача, оскільки тепер просто запихаємо повідомлення у чергу.

Розподіл між вузлами:

logging-1

```
INFO    Logging for application startup.
[LOGGING] Connected to Hazelcast cluster!
[LOGGING] Successfully registered http://logging1:8000 to config-server
INFO    Application startup complete.
INFO    Uvicorn running on http://0.0.0.0:8000 (Press CTRL+C to quit)
INFO    172.18.0.10:41716 - "GET /health HTTP/1.1" 200
[LOGGING] Stored transaction 1776590730744523465 for user 'user'
INFO    172.18.0.10:53826 - "POST /transaction HTTP/1.1" 200
[LOGGING] Stored transaction 1776590826581112759 for user 'user'
INFO    172.18.0.10:47690 - "POST /transaction HTTP/1.1" 200
[LOGGING] Stored transaction 1776590878472449547 for user 'user'
INFO    172.18.0.10:44700 - "POST /transaction HTTP/1.1" 200
[LOGGING] Stored transaction 1776590918794465719 for user 'user'
INFO    172.18.0.10:53432 - "POST /transaction HTTP/1.1" 200
[LOGGING] Stored transaction 1776590969560479173 for user 'user'
INFO    172.18.0.10:49526 - "POST /transaction HTTP/1.1" 200
```

logging-2

```
INFO    Logging for application startup.
[LOGGING] Connected to Hazelcast cluster!
[LOGGING] Successfully registered http://logging2:8000 to config-server
INFO    Application startup complete.
INFO    Uvicorn running on http://0.0.0.0:8000 (Press CTRL+C to quit)
INFO    172.18.0.10:55152 - "GET /health HTTP/1.1" 200
[LOGGING] Stored transaction 1776590787458214547 for user 'user'
INFO    172.18.0.10:55776 - "POST /transaction HTTP/1.1" 200
[LOGGING] Stored transaction 1776591010806038261 for user 'user'
INFO    172.18.0.10:41526 - "POST /transaction HTTP/1.1" 200
```

logging-3

```
[LOGGING] Connected to Hazelcast cluster!
[LOGGING] Successfully registered http://logging3:8000 to config-server
INFO    Application startup complete.
INFO    Uvicorn running on http://0.0.0.0:8000 (Press CTRL+C to quit)
INFO    172.18.0.10:43688 - "GET /health HTTP/1.1" 200
[LOGGING] Stored transaction 1776590899231783501 for user 'user'
INFO    172.18.0.10:34226 - "POST /transaction HTTP/1.1" 200
[LOGGING] Stored transaction 1776590939634532089 for user 'user'
INFO    172.18.0.10:34294 - "POST /transaction HTTP/1.1" 200
[LOGGING] Stored transaction 1776590989161231209 for user 'user'
INFO    172.18.0.10:37616 - "POST /transaction HTTP/1.1" 200
```

З логів logging сервісів бачимо, що транзакції були успішно розподілені.

Перевірка через GET

Curl

```
curl -X 'GET' \
'http://localhost:8080/user/user' \
-H 'accept: application/json'
```

Request URL

```
http://localhost:8080/user/user
```

Server response

Code	Details
200	Response body <pre>{ "balance": 314.5, "transactions": [{ "user_id": "user", "amount": -150, "transaction_id": "1776590989161231209" }, { "user_id": "user", "amount": 100, "transaction_id": "1776590918794465719" }, { "user_id": "user", "amount": -6.2, "transaction_id": "1776590826581112759" }, { "user_id": "user", "amount": 10.2, "transaction_id": "1776590730744523465" }, { "user_id": "user" }] }</pre>

```
{
  "user_id": "user",
  "amount": -50,
  "transaction_id": "1776590939634532089"
},
{
  "user_id": "user",
  "amount": 66,
  "transaction_id": "1776590787458214547"
},
{
  "user_id": "user",
  "amount": 1.5,
  "transaction_id": "1776591010806038261"
},
{
  "user_id": "user",
  "amount": -20,
  "transaction_id": "1776590878472449547"
},
{
  "user_id": "user",
  "amount": 33,
  "transaction_id": "1776590899231783501"
}
]
```

```
},
{
  "user_id": "user",
  "amount": 330,
  "transaction_id": "1776590969560479173"
}
]
```

Якщо бути хорошим математиком, або просто взяти калькулятор, то можна зауважити що математика тут математикує.

Також працюють запити до ендпоінту /accounts, а справа логи фасаду, де видно всі POST та GET запити:

Curl

```
curl -X 'GET' \
'http://localhost:8080/accounts' \
-H 'accept: application/json'
```

Request URL

```
http://localhost:8080/accounts
```

Server response

Code	Details
200	Response body <pre>{ "user": 314.5 }</pre>

```
INFO 192.168.65.1:35612 - "GET /docs HTTP/1.1" 200
INFO 192.168.65.1:35612 - "GET /openapi.json HTTP/1.1" 200
INFO 192.168.65.1:62149 - "GET /health HTTP/1.1" 200
[FACADE] Received request from user 'user' for amount 10.2
INFO 192.168.65.1:53137 - "POST /process HTTP/1.1" 200
[FACADE] Received request from user 'user' for amount 66.0
INFO 192.168.65.1:52633 - "POST /process HTTP/1.1" 200
[FACADE] Received request from user 'user' for amount -6.2
INFO 192.168.65.1:34079 - "POST /process HTTP/1.1" 200
[FACADE] Received request from user 'user' for amount -20.0
INFO 192.168.65.1:34015 - "POST /process HTTP/1.1" 200
[FACADE] Received request from user 'user' for amount 33.0
INFO 192.168.65.1:50753 - "POST /process HTTP/1.1" 200
[FACADE] Received request from user 'user' for amount 100.0
INFO 192.168.65.1:44255 - "POST /process HTTP/1.1" 200
[FACADE] Received request from user 'user' for amount -50.0
INFO 192.168.65.1:62937 - "POST /process HTTP/1.1" 200
[FACADE] Received request from user 'user' for amount 330.0
INFO 192.168.65.1:63024 - "POST /process HTTP/1.1" 200
[FACADE] Received request from user 'user' for amount -150.0
INFO 192.168.65.1:18707 - "POST /process HTTP/1.1" 200
[FACADE] Received request from user 'user' for amount 1.5
INFO 192.168.65.1:43260 - "POST /process HTTP/1.1" 200
INFO 192.168.65.1:43585 - "GET /user/user HTTP/1.1" 200
INFO 192.168.65.1:22845 - "GET /accounts HTTP/1.1" 200
INFO 192.168.65.1:24938 - "GET /user/user HTTP/1.1" 200
INFO 192.168.65.1:52669 - "GET /user/user HTTP/1.1" 200
```

З логів counter сервісу бачимо як послідовно мінявся баланс користувача:

```
[COUNTER] Successfully connected to PostgreSQL!
[COUNTER] Registered with config server
INFO Uvicorn running on http://0.0.0.0:8000 (Press CTRL+C to quit)
Topic transactions not found in cluster metadata
Group Coordinator Request failed: [Error 15] GroupCoordinatorNotAvailableError
Marking the coordinator dead (node 1)for group counter-group.
[COUNTER] Kafka Consumer started listening to 'transactions' topic
INFO 172.18.0.10:40844 - "GET /health HTTP/1.1" 200
[COUNTER] Processed Kafka msg. User 'user' applied 10.2. New balance: 10.20
[COUNTER] Processed Kafka msg. User 'user' applied 66.0. New balance: 76.20
[COUNTER] Processed Kafka msg. User 'user' applied -6.2. New balance: 70.00
[COUNTER] Processed Kafka msg. User 'user' applied -20.0. New balance: 50.00
[COUNTER] Processed Kafka msg. User 'user' applied 33.0. New balance: 83.00
[COUNTER] Processed Kafka msg. User 'user' applied 100.0. New balance: 183.00
[COUNTER] Processed Kafka msg. User 'user' applied -50.0. New balance: 133.00
[COUNTER] Processed Kafka msg. User 'user' applied 330.0. New balance: 463.00
[COUNTER] Processed Kafka msg. User 'user' applied -150.0. New balance: 313.00
[COUNTER] Processed Kafka msg. User 'user' applied 1.5. New balance: 314.50
INFO 172.18.0.10:40682 - "GET /balance/user HTTP/1.1" 200
INFO 172.18.0.10:39402 - "GET /balance HTTP/1.1" 200
```


Перевірка Відмовостійкості

```
docker pause hws-counter_service-1
```

Справді впав (ліво) але транзакція на user_check була успішна (право)

Curl

```
curl -X 'GET' \
  'http://localhost:8080/health' \
  -H 'accept: application/json'
```

Request URL

```
http://localhost:8080/health
```

Server response

Code	Details
200	Response body <pre>{ "status": "degraded", "config_server": "ok", "kafka": "ok", "counter_services": { "http://counter_service:8000": "down" }, "logging_services": { "http://logging3:8000": { "status": "ok", "hazelcast": "connected" }, "http://logging1:8000": { "status": "ok", "hazelcast": "connected" }, "http://logging2:8000": { "status": "ok", "hazelcast": "connected" } } }</pre>

Curl

```
curl -X 'POST' \
  'http://localhost:8080/process' \
  -H 'accept: application/json' \
  -H 'Content-Type: application/json' \
  -d '{
    "user_id": "user_check",
    "amount": 20
  }'
```

Request URL

```
http://localhost:8080/process
```

Server response

Code	Details
200	Response body <pre>{ "transaction_id": "1776592817675356667", "status": "Message queued successfully" }</pre>

Тільки транзакція зараз у черзі і чекає поки counter почне її обробляти:

Curl

```
curl -X 'GET' \
  'http://localhost:8080/user/user_check' \
  -H 'accept: application/json'
```

Request URL

```
http://localhost:8080/user/user_check
```

Server response

Code	Details
200	Response body <pre>{ "balance": null, "transactions": [{ "user_id": "user_check", "amount": 20, "transaction_id": "1776592817675356667" }] }</pre>

Curl

```
curl -X 'GET' \
  'http://localhost:8080/accounts' \
  -H 'accept: application/json'
```

Request URL

```
http://localhost:8080/accounts
```

Server response

Code	Details
200	Response body <pre>null</pre>

Спробуємо відновити його роботу:

```
docker unpause hws-counter_service-1
```


І тепер все працює як треба:

Curl

```
curl -X 'GET' \
  'http://localhost:8080/user/user_check' \
  -H 'accept: application/json'
```

Request URL

```
http://localhost:8080/user/user_check
```

Server response

Code	Details
200	Response body <pre>{ "balance": 20, "transactions": [{ "user_id": "user_check", "amount": 20, "transaction_id": "1776592817675356667" }] }</pre>

Curl

```
curl -X 'GET' \
  'http://localhost:8080/accounts' \
  -H 'accept: application/json'
```

Request URL

```
http://localhost:8080/accounts
```

Server response

Code	Details
200	Response body <pre>{ "user": 314.5, "user_check": 20 }</pre> Response headers <pre>content-length: 32 content-type: application/json date: Sun, 19 Apr 2026 19:05:33 GMT server: uvicorn</pre>

А нижче тут логи з сервісу підрахунків (показує що він на паузі) і також з фасаду, де є останні запити до user_check.

counter:

```
Failed fetch messages from 1: [Error 7] RequestTimeoutError
Heartbeat failed: local member_id was not recognized; resetting and re-joining group
Heartbeat session expired - marking coordinator dead
Marking the coordinator dead (node 1)for group counter-group.
OffsetCommit failed for group counter-group due to group error ([Error 25] UnknownMemberIdError: counter-group), will rejoin
Auto offset commit failed: [Error 25] UnknownMemberIdError: counter-group
[COUNTER] Processed Kafka msg. User 'user_check' applied 20.0. New balance: 20.00
INFO 172.18.0.10:36486 - "GET /balance/user_check HTTP/1.1" 200
INFO 172.18.0.10:33698 - "GET /balance HTTP/1.1" 200
```

facade

```
[FACADE] Received request from user 'user_check' for amount 20.0
INFO    192.168.65.1:36156 - "POST /process HTTP/1.1" 200
INFO    192.168.65.1:30245 - "GET /health HTTP/1.1" 200
INFO    192.168.65.1:65056 - "GET /user/user_check HTTP/1.1" 200
INFO    192.168.65.1:26072 - "GET /accounts HTTP/1.1" 200
INFO    192.168.65.1:23889 - "GET /docs HTTP/1.1" 200
INFO    192.168.65.1:23889 - "GET /openapi.json HTTP/1.1" 200
INFO    192.168.65.1:49715 - "GET /user/user_check HTTP/1.1" 200
INFO    192.168.65.1:38545 - "GET /accounts HTTP/1.1" 200
```

Висновок

В цій лабі я знову познайомився з тим як працює асинхронна черга Kafka (брали на курсі з Обробки Великих Даних) і зміг під'єднати до нашої мікросервісної архітектури. Тепер вона виглядає набагато ефектніше ніж в першій домашній.